

Buffer allocation — the defensible core

What to be able to say, and why each part is true

RailCast · 8 September 2026

What this document is for. The three-part argument in section 3 is the whole proof that this is a formulation rather than a library call. It is what an interviewer will probe, and it is the part that must be reproducible on a whiteboard without notes. Everything else here is context for it.

1. The claim, in one paragraph

A timetable contains padding: scheduled running times exceed the minimum a train can actually achieve. That padding absorbs delay. It is currently distributed however the timetable happens to distribute it. *Given the same total padding*, is there a better allocation — one that reduces expected lateness?

This is posed as a linear program: the padding on each segment is a decision variable, the total is a budget constraint, delay propagation along the route is a set of linear constraints, and expected lateness across historical days is the objective. It is solved with HiGHS via `scipy.optimize.linprog`. The formulation is derived below; the solver is only the arithmetic.

2. The program

Stops $i = 0, 1, \dots, n$ along one train's route; segment i runs from stop $i - 1$ to stop i . Scenarios $\omega \in \Omega$ are historical days.

Data

$$\begin{aligned} m_i &= \text{minimum technical running time on segment } i \\ r_i &= \text{scheduled running time on segment } i \\ b_i^0 &= r_i - m_i \\ B &= \sum_{i=1}^n b_i^0 \\ \delta_i^\omega &= \max(0, \rho_i^\omega - m_i) \end{aligned}$$

ρ_i^ω is the observed running time on segment i on day ω ; m_i is estimated as the 5th percentile of observed running times; b_i^0 is the timetable's own allocation and is the baseline; B is the budget, so the same total is redistributed and journey time is unchanged; δ_i^ω is the primary delay actually incurred.

Variables, objective, constraints

$$b_i \geq 0 \quad (i = 1 \dots n), \quad L_i^\omega \geq 0 \quad (i = 1 \dots n, \omega \in \Omega)$$

$$\min_{b, L} \quad \frac{1}{|\Omega|} \sum_{\omega \in \Omega} \sum_{i=1}^n w_i L_i^\omega$$

$$\begin{aligned}
(\text{C1}) \quad L_i^\omega &\geq L_{i-1}^\omega + \delta_i^\omega - b_i & \forall i, \forall \omega \\
(\text{C2}) \quad L_i^\omega &\geq 0 & \forall i, \forall \omega \\
(\text{C3}) \quad \sum_{i=1}^n b_i &\leq B \\
(\text{C4}) \quad 0 \leq b_i &\leq \bar{b}_i & \forall i \\
(\text{C5}) \quad L_0^\omega &= \ell_0^\omega
\end{aligned}$$

3. Why the maximum can be replaced by two inequalities

This is the argument to know cold.

The problem

Delay propagation along a route is genuinely nonlinear:

$$L_i^\omega = \max(0, L_{i-1}^\omega + \delta_i^\omega - b_i).$$

A train carries its delay forward; buffer eats into it; and it cannot become “negatively late” in a way that lets it depart a scheduled stop early. That max cannot appear in a linear program.

Constraints (C1) and (C2) are the same expression written as two *inequalities*. They say L_i^ω is at least each of the two arguments — which is weaker than saying it equals their maximum. Every feasible point of the true system is feasible here, but so are points where L_i^ω is too large. The relaxation must be shown to be **tight**: that the optimum of the relaxed program is a solution of the true recursion.

The argument, in three steps

- 1. The objective pushes every L_i^ω down.** Each enters with coefficient $w_i/|\Omega| \geq 0$. Nothing in the program rewards a larger L_i^ω .
- 2. Its only lower bounds are (C1) and (C2).** No other constraint mentions L_i^ω from below. Driven down as far as feasibility allows, it comes to rest exactly at $\max(0, L_{i-1}^\omega + \delta_i^\omega - b_i)$ — the value the true recursion specifies.
- 3. Reducing it never costs anything elsewhere.** L_i^ω appears once more, on the right-hand side of (C1) for index $i+1$, with coefficient $+1$. Reducing L_i^ω *lowers* that bound, which *relaxes* the next constraint. Both forces act in the same direction. There is no configuration in which inflating L_i^ω buys a saving.

The formal version

Claim. Let (b^*, L^*) be optimal. Then $L_i^{*\omega} = \max(0, L_{i-1}^{*\omega} + \delta_i^\omega - b_i^*)$ for every i, ω .

Proof. Suppose some $L_i^{*\omega}$ strictly exceeds that maximum. Lower it to the maximum. Then (C1) at index i still holds, since the value still meets the bound; (C2) still holds, since the maximum is non-negative; (C1) at index $i+1$ has a strictly *smaller* right-hand side and so still holds; and (C3), (C4) are untouched, involving only b . The point remains feasible, and the objective has strictly decreased — contradicting optimality.

□

The gap in that proof, and the condition that closes it

The last step needs the objective to *strictly* decrease, which needs $w_i > 0$. Under terminus-only weighting ($w_i = 0$ for every $i < n$) that fails, and the naive argument does not go through.

The relaxation is still tight, for a subtler reason. Lowering L_i^ω relaxes (C1) at $i + 1$, which lets L_{i+1}^ω fall, which relaxes (C1) at $i + 2$, and so on: a reduction propagates forward along the chain with coefficient 1 at every step. So it reaches the terminus, where the weight is positive, and the objective does strictly decrease.

The general condition. The relaxation is tight at (i, ω) provided

$$w_j \geq 0 \text{ for all } j, \quad \text{and} \quad \sum_{j \geq i} w_j > 0.$$

That is: every stop from i onward has non-negative weight, and at least one of them is strictly positive. Under uniform weighting every index qualifies immediately. Under terminus-only weighting every index qualifies because $w_n > 0$ and the reduction propagates forward to it.

This is the version to give if pushed. Stating only “the objective pushes it down” invites the follow-up “*what if that stop has zero weight?*”, and terminus-only weighting — which this project runs — is exactly that case.

4. The failure mode

Introduce a negative weight anywhere and the exactness breaks.

Suppose $w_k < 0$ at some stop, intended to reward early arrival. Then the objective *rewards* a larger L_k^ω . The solver will inflate it — feasibly, since (C1) and (C2) are only lower bounds — to collect that reward, and the constraint at $k + 1$ absorbs the inflation by raising L_{k+1}^ω , which may be cheap if the weights beyond k are small.

The lateness variables then no longer represent lateness. The reported objective is not the expected lateness of any timetable, and the buffer vector is optimal for a problem nobody posed.

The fix is not a negative weight. Rewarding earliness requires representing it as a separate non-negative variable — $E_i^\omega \geq (\text{scheduled}) - (\text{actual})$, carrying its own non-negative cost — so that both quantities are minimised and neither is inflated for gain.

5. Three methodological decisions

1. Significance is a paired bootstrap over *days*, not over rows

The plan says to evaluate against b^0 on held-out days. It does not say how to decide the difference is real, and with a handful of days an improvement of a few seconds is easily noise. The project already has the apparatus: the paired bootstrap veto used for model promotion.

One refinement matters. That test resamples *rows*. Here the independent unit is the **day**, not the stop-arrival: within one day, delay at consecutive stops is strongly correlated by construction — that is the propagation the whole model describes. Resampling stop-arrivals would treat correlated observations as independent and understate the variance, producing a confident answer that is not warranted.

The number that governs the statistics is the number of days, not the number of variables. A 29-stop service with 32 complete days gives 896 lateness variables and **32** independent scenarios. Power comes from 32. Resample whole days, with replacement, and report the interval on the difference $(\text{cost}(b^*) - \text{cost}(b^0))$.

Consequence to accept in advance: 32 scenarios is a small sample for a significance claim, and the honest outcome may be “an improvement was measured and cannot be distinguished from noise.” The way to raise power is more days, and the data source supports it — the API serves history back to 2007, so the scenario set can be extended by backfilling more dates rather than by weakening the test.

2. The cap $\bar{b}_i$ needs defining, and reporting

$\bar{b}_i$ appears in (C4) and is nowhere defined. Two separate questions:

Is it needed mathematically? No. Buffer is scarce — (C3) binds — so the solver has no reason to place buffer where it cannot be used. Allocating more than $\max_{\omega}(L_{i-1}^{\omega} + \delta_i^{\omega})$ to segment i is provably wasted, because beyond that point the segment already absorbs everything reaching it in every scenario, and the budget would be better spent elsewhere.

Is it needed operationally? Yes, if the answer is meant to be implementable. Without it the optimum may schedule a four-minute segment at eleven minutes, which conflicts with other traffic and is not a timetable anyone would adopt. A defensible choice is a proportional cap, $\bar{b}_i = \alpha m_i$, in the spirit of the running-time supplements timetable planners already use.

Report whether the cap binds. If it does not, it is inert and the result is data-driven. If it does, the result is partly driven by a chosen constant and that must be stated, along with the sensitivity to α . Run once uncapped as the reference.

3. Report both weightings; do not choose one

w_i is a modelling decision and boardings are unavailable. Choosing a plausible-looking weight vector would reintroduce precisely the objection that ruled out the delay-management formulation: an objective function containing invented numbers.

Run both defensible extremes and publish both:

Weighting	w_i	Optimises for
Terminus-only	$w_n = 1$, else 0	end-to-end punctuality
Uniform	$w_i = 1$ for all i	passengers alighting anywhere

They bracket the answer. If the optimised allocation beats b^0 under both, the conclusion is robust to the weighting — a stronger claim than either result alone. If it wins under one and loses under the other, that is a finding about where the gain comes from, and it is reported as such rather than resolved by picking the flattering one.

6. Questions to expect

1. *How is this optimization rather than calling a library?* — Answer with the formulation: variables, objective, constraints, and the linearisation of the max. The solver is arithmetic; the modelling is section 3.

2. *Your constraints are inequalities but the dynamics are an equality. Why is that valid?* — The three-step argument, then the $\sum_{j \geq i} w_j > 0$ condition.
 3. *What breaks it?* — A negative weight. Explain the inflation mechanism.
 4. *Why is this a linear program and not an integer program?* — Every variable is continuous; there is no discrete decision. Delay management would be an integer program, because hold-or-drop is binary — and it was rejected for lack of data, not for difficulty.
 5. *How do you know the improvement is real?* — Paired bootstrap over days, and the sample size is days.
 6. *What are its assumptions?* — Primary delays exogenous (the largest threat: drivers pace to the timetable, so added buffer may attract a slower run); minimum running time estimated from a sample quantile; single train, no network capacity; counterfactual, so simulated rather than validated.
 7. *Does it assume segments are independent?* — No. Scenarios are whole days, so within-day correlation is preserved exactly. This is a genuine advantage of sample average approximation over a parametric model.
-

If the optimiser does not beat the existing timetable, that result is recorded and kept. A negative result honestly measured is worth more than a claim that has to be hedged.